Лабораторная работа № 3 по курсу дискретного анализа: Исследование качества программ

Выполнил студент группы М8О-203БВ-24 *Пятницкий Артём*.

Условие

Для реализации словаря из предыдущей лабораторной работы, необходимо провести исследование скорости выполнения и потребления оперативной памяти. В случае выявления ошибок или явных недочётов, требуется их исправить.

Результатом лабораторной работы является отчёт, состоящий из:

- Дневника выполнения работы, в котором отражено что и когда делалось, какие средства использовались и какие результаты были достигнуты на каждом шаге выполнения лабораторной работы.
- Выводов о найденных недочётах.
- Сравнение работы исправленной программы с предыдущей версией.
- Общих выводов о выполнении лабораторной работы, полученном опыте.

Минимальный набор используемых средств должен содержать утилиту ***gprof*** и библиотеку ***dmalloc***, однако их можно заменять на любые другие аналогичные или более развитые утилиты (например, Valgrind или Shark) или добавлять к ним новые (например, gcov).

Дневник выполнения работы

В первую очередь программа была протестирована на утечки памяти с помощью санитайзеров (**Sanitizers)** - встроенные в компилятор GCC инструменты, которые позволяют находить большинство ошибок работы с памятью (утечки, выход за границы массива, использование памяти после освобождения) и неопределённым поведением (UB). Чтобы санитайзеры могли показать нам, на какой конкретно строке кода произошла ошибка, компилятору нужно задать флаг «-g» generate/debug (генерация/отладка). Этот флаг позволяет отладчику сопоставлять машинный код с исходным кодом: видеть имена функций, переменных и номера строк. Также для включения самих санитайзеров нужно указать флаг «-fsanitize=address,undefined».

Перекомпилируем программу с использованием флагов:

```
cmake_minimum_required(VERSION 3.10)
project(PATRICIA LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -g -Wall -Wextra -pedantic -
fsanitize=address,undefined")
set(CMAKE_EXE_LINKER_FLAGS "${CMAKE_EXE_LINKER_FLAGS} -
fsanitize=address,undefined")

add_library(${PROJECT_NAME}_lib src/patricia.cpp)

add_executable(${PROJECT_NAME} src/main.cpp)
target_link_libraries(${PROJECT_NAME} PRIVATE ${PROJECT_NAME}_lib)
```

После успешной компиляции, необходимо запустить программу на тестовом вводе. В качестве тестового ввода, использовался файл на 1 миллион элементов, с равномерным распределением команд вставки, поиска и удаления.

```
./PATRICIA  < ../Discrete-analysis-Labs/Laboratory_2/tests/test_mixed_1m.txt
```

В результате выполнения, никаких ошибок не было найдено.

Следующим этапом проверки является **Valgrind (Memcheck)** – это более тяжёлый инструмент, в отличии от санитайзеров, он выполняет код в виртуальной машине и может найти, что пропустили санитайзеры (например, использование неинициализированной памяти). Важным нюансом является конфликт Valgrind и AddressSanitizer, они делают похожую работу, но AddressSanitizer модифицирует код на этапе компиляции, а Valgrind выполняет код в виртуальной машине, не меняя исходный код. Если запустить скомпилированную с AddressSanitizer программу под Valgrind, то будет множество ложных ошибок. Поэтому, перед запуском Valgrind нужно убрать флаги санитайзера, но оставить отладочные символы «-g», перекомпилировать программу и запустить Valgrind.

Для подробного отчёта будем использовать флаги:

- «--leak-check=full» - подробный отчёт об утечках памяти

- «--show-leak-kinds=all» — показывать все виды утечек (явно потерянные блоки, блоки, на которые остались скрытые указатели, и т.д.).
- «--track-origins=yes» — если Valgrind найдет использование *неинициализированной* переменной

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes
./PATRICIA < ../Discrete-analysis-Labs/Laboratory_2/tests/test_mixed_1m.txt >
/dev/null
```

В результате получим вывод с отчётом

```
==522175== Memcheck, a memory error detector
==522175== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==522175== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==522175== Command: ./PATRICIA
==522175==
==522175==
==522175== HEAP SUMMARY:
==522175== in use at exit: 0 bytes in 0 blocks
==522175== total heap usage: 1,166,670 allocs, 1,166,670 frees, 74,748,608 bytes
allocated
==522175==
==522175== All heap blocks were freed -- no leaks are possible
==522175==
==522175== For lists of detected and suppressed errors, rerun with: -s
==522175== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Из отчёта видно, что ошибок утечек памяти не было найдено, что свидетельствует о корректном управлении памятью в программе.

Проведём профилирование потребление кучи. Для анализа динамики потребления памяти в процессе используется флаг «--tool=massif» , а также флаг «--massif-out-file=massif.out» для сохранения отчёта в файл «massif.out».

```
valgrind --tool=massif --massif-out-file=massif.out ./PATRICIA <
../tests/test_1m.txt > /dev/null
```

```
ms_print massif.out
```

В результате получим отчёт потребления памяти программой:

```
--------------------------------------------------------------------------------
Command:            ./PATRICIA
Massif arguments:   --massif-out-file=massif.out
ms_print arguments: massif.out
--------------------------------------------------------------------------------


    MB
68.74^                                                       #
     |                                   ::::::::::::::::#:
     |                                 ::::              #:::
     |                               ::::::              #::::
     |                             ::::::::              #::::::
     |                           @@::::::::              #::::::::
     |                         ::@ ::::::::              #:::::::::
     |                        :::@ ::::::::              #::::::::::
     |                      ::@:::@ ::::::::             #::::::::::@
     |                     :::@:::@ ::::::::             #::::::::::@::
     |                  ::::::@:::@ ::::::::             #::::::::::@::::::::
     |                @::::::@:::@ ::::::::             #::::::::::@:::    :
     |              :::@::::::@:::@ ::::::::             #::::::::::@:::    :
     |             ::::@::::::@:::@ ::::::::             #::::::::::@:::    :
     |           @:::::@::::::@:::@ ::::::::             #::::::::::@:::    :
     |         ::@:::::@::::::@:::@ ::::::::             #::::::::::@:::    :
     |        :::@:::::@::::::@:::@ ::::::::             #::::::::::@:::    :
     |     ::::::@:::::@::::::@:::@ ::::::::             #::::::::::@:::    :
     |   ::::::::@:::::@::::::@:::@ ::::::::             #::::::::::@:::    :
     | @@::::::::@:::::@::::::@:::@ ::::::::             #::::::::::@:::    :
   0 +------------------------------------------------------------------------>Gi
     0                                                                   25.52

Number of snapshots: 65
 Detailed snapshots: [2, 3, 13, 19, 26, 31, 41 (peak), 56]
```

График потребления памяти, который демонстрирует линейный рост (происходит добавление элементов), фиксированный промежуток (процесс поиска элементов) и линейное падение потребления (удаление 50% элементов).

```
--------------------------------------------------------------------------------
  n        time(i)         total(B)   useful-heap(B) extra-heap(B)    stacks(B)
--------------------------------------------------------------------------------
  4  1,516,206,383        8,056,232        7,170,176       886,056            0
  5  1,814,220,050        9,566,792        8,512,896     1,053,896            0
  6  2,192,727,118       11,535,488       10,262,848     1,272,640            0
  7  2,502,383,283       13,127,120       11,677,632     1,449,488            0
  8  2,720,256,565       14,225,048       12,653,568     1,571,480            0
  9  3,092,865,078       16,125,992       14,343,296     1,782,696            0
 10  3,408,127,268       17,713,016       15,753,984     1,959,032            0
 11  3,666,052,186       18,989,720       16,888,832     2,100,888            0
 12  4,141,790,833       21,454,280       19,079,552     2,374,728            0
 13  4,554,309,699       23,560,856       20,952,064     2,608,792            0
```

```
--------------------------------------------------------------------------------
  n        time(i)         total(B)   useful-heap(B) extra-heap(B)    stacks(B)
--------------------------------------------------------------------------------
 14  4,863,711,948       25,142,480       22,357,952     2,784,528            0
 15  5,224,675,092       26,960,048       23,973,568     2,986,480            0
 16  5,688,787,627       29,254,904       26,013,440     3,241,464            0
 17  6,049,750,065       31,038,272       27,598,656     3,439,616            0
 18  6,256,022,707       32,058,800       28,505,792     3,553,008            0
 19  6,616,981,212       33,826,544       30,077,120     3,749,424            0
88.92% (30,077,120B) (heap allocation functions) malloc/new/new[], --alloc-fns, etc.
```

```
--------------------------------------------------------------------------------
  n        time(i)         total(B)   useful-heap(B) extra-heap(B)    stacks(B)
--------------------------------------------------------------------------------
 20  7,132,597,711       36,327,968       32,300,608     4,027,360            0
 21  7,390,401,737       37,559,816       33,395,584     4,164,232            0
 22  7,734,154,729       39,301,424       34,943,680     4,357,744            0
 23  8,249,779,809       41,895,224       37,249,280     4,645,944            0
 24  8,593,550,787       43,613,720       38,776,832     4,836,888            0
 25  8,851,376,342       44,883,152       39,905,216     4,977,936            0
 26  9,195,143,063       46,582,208       41,415,488     5,166,720            0
```

```
--------------------------------------------------------------------------------
  n        time(i)         total(B)   useful-heap(B) extra-heap(B)    stacks(B)
--------------------------------------------------------------------------------
 27  9,538,884,120       48,308,192       42,949,696     5,358,496            0
 28  9,882,647,471       50,043,248       44,491,968     5,551,280            0
 29 10,140,459,638       51,339,104       45,643,840     5,695,264            0
 30 10,484,197,142       53,057,456       47,171,264     5,886,192            0
 31 10,913,902,722       55,184,264       49,061,760     6,122,504            0
```

n	time(i)	total(B)	useful-heap(B)	extra-heap(B)	stacks(B)
32	11,429,643,787	57,710,456	51,307,264	6,403,192	0
33	11,816,453,856	59,582,312	52,971,136	6,611,176	0
34	12,074,335,128	60,825,176	54,075,904	6,749,272	0
35	12,332,213,507	62,091,368	55,201,408	6,889,960	0
36	12,719,023,600	64,005,128	56,902,528	7,102,600	0
37	13,105,828,712	65,903,840	58,590,272	7,313,568	0
38	13,363,697,532	67,171,472	59,717,056	7,454,416	0
39	13,750,501,949	69,043,328	61,380,928	7,662,400	0
40	14,266,239,554	71,539,496	63,599,744	7,939,752	0
41	19,612,555,756	72,081,944	64,081,920	8,000,024	0
42	20,128,285,111	69,036,416	61,374,784	7,661,632	0
43	20,515,096,157	66,741,632	59,334,976	7,406,656	0
44	20,772,975,864	65,206,592	57,970,496	7,236,096	0
45	21,288,709,457	62,152,208	55,255,488	6,896,720	0
46	21,675,505,247	59,856,992	53,215,296	6,641,696	0
47	21,933,378,751	58,326,416	51,854,784	6,471,632	0
48	22,320,176,651	56,032,568	49,815,808	6,216,760	0
49	22,578,051,062	54,514,808	48,466,688	6,048,120	0
50	22,784,330,174	53,276,336	47,365,824	5,910,512	0
51	22,990,603,501	52,016,624	46,246,080	5,770,544	0
52	23,196,884,424	50,718,248	45,091,968	5,626,280	0
53	23,403,165,656	49,450,688	43,965,248	5,485,440	0
54	23,609,449,981	48,160,952	42,818,816	5,342,136	0
55	23,815,728,528	46,882,808	41,682,688	5,200,120	0
56	24,022,004,383	45,616,112	40,556,736	5,059,376	0

n	time(i)	total(B)	useful-heap(B)	extra-heap(B)	stacks(B)
57	24,228,284,133	44,361,584	39,441,600	4,919,984	0
58	24,434,557,896	43,115,768	38,334,208	4,781,560	0
59	24,640,832,013	41,858,072	37,216,256	4,641,816	0
60	24,847,110,847	40,615,352	36,111,616	4,503,736	0
61	25,053,395,728	39,354,632	34,990,976	4,363,656	0
62	25,259,669,082	38,114,936	33,889,024	4,225,912	0
63	25,465,949,050	36,850,976	32,765,504	4,085,472	0
64	27,406,438,854	36,081,872	32,081,856	4,000,016	0

Данные таблиц подтверждают выводы, сделанные по графику. С 1 по 41 строку наблюдается линейный рост, обусловленный созданием новых узлов (вставкой). В строке 41 видено пиковое потребление памяти. А между 40 и 41 строчкой, как раз таки происходит поиск (происходит 5 миллиардов инструкций, хотя рост памяти не наблюдается – это и есть процесс поиска). С 42 строчки память начинает линейно уменьшаться, происходит процесс

удаления элементов (удаляются 50 % вставленных). После чего программа завершает своё выполнение, деструктор очищает все оставшиеся узлы, потребление памяти резко падает.

Следующим этапом является профилирование производительности с помощью инструмента «perf». Важное правило профилирования: «никогда не профилировать неоптимизированную программу». По умолчанию компилятор не делает никакой оптимизации (уровень «-O0»), поэтому программа работает медленно из-за лишних обращений к памяти и т. д. Поэтому если мы используем профилирование с «-O0», будут найдены проблемы не самого кода, а того, что компилятор не оптимизировал код. Поэтому нам надо пересобрать проект с флагом «-O3»:

```
set(CMAKE_CXX_FLAGS "-O3 -g -Wall -Wextra -pedantic")
```

Для начала получим общую статистику, в который будут собраны данные о том, сколько времени процессор был занят программой, сколько раз ядро ОС прерывало нашу программу, переходы с одного ядра на другое, сколько тактов процессора было потрачено и тд.

Запустим многократный запуск программы с 40 прогонами с усреднением результатов:

```
perf stat -r40 ./PATRICIA < ../tests/test_mixed_5m.txt > /dev/null
```

В результате получим следующий вывод:

```
Performance counter stats for 'bash -lc for i in $(seq 1 8); do ./build/PATRICIA <
tests/test_1m.txt > /dev/null; done' (40 runs):


        3966757626      task-clock:u                     #    0.394 CPUs utilized
( +-  0.18% )
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
            158961      page-faults:u                    #   40.073 K/sec
( +-  0.00% )


            10.080 +- 0.110 seconds time elapsed  ( +-  1.09% )
```

Теперь мы должны записать профиль. Утилита «perf» работает по принципу сэмплирования: он тысячи раз в секунду прерывает программу и смотрит, в какой функции сейчас находится процессор.

Запускаем утилиту perf:

```
perf record -g -- ./PATRICIA < ../Discrete-analysis-Labs/Laboratory_2/tests/test_mixed_1m.txt > /dev/null
```

```
[ perf record: Woken up 3 times to write data ]
[ perf record: Captured and wrote 0,753 MB perf.data (8514 samples) ]
```

Откроем файл «perf.data»

```
perf report -g
```

В открывшемся интерактивном интерфейсе, сможем увидеть доли времени, проведённой в конкретной функции в процентах.

```
Samples: 8K of event 'task-clock:upppH', Event count (approx.): 2128500000
  Children      Self  Command   Shared Object       Symbol
-   85,10%    13,25%  PATRICIA  PATRICIA            [.] main
   - 71,85% main
      + 37,10% std::basic_istream<char, std::char_traits<char> >& std::operator>><char, std::char_traits<char>, std::allocator<char> >(std::basic_istream<char, std::char_traits
      + 24,11% std::istream& std::istream::_M_extract<unsigned long>(unsigned long&)
      + 2,36% std::ostream& std::ostream::_M_insert<unsigned long>(unsigned long)
      + 2,35% std::basic_ostream<char, std::char_traits<char> >& std::operator<< <std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&, char const*)
        1,70% __gnu_cxx::stdio_sync_filebuf<char, std::char_traits<char> >::underflow()
        0,82% ungetc
        0,82% std::basic_ostream<char, std::char_traits<char> >& std::__ostream_insert<char, std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&, char c
        0,66% __gnu_cxx::stdio_sync_filebuf<char, std::char_traits<char> >::uflow()
   + 12,61% 0x9800000000000011
-   47,44%     0,00%  PATRICIA  [unknown]            [.] 0x0000000000000009
   - 0x9
     - 37,55% main
        + 24,11% std::istream& std::istream::_M_extract<unsigned long>(unsigned long&)
        + 10,03% std::basic_istream<char, std::char_traits<char> >& std::operator>><char, std::char_traits<char>, std::allocator<char> >(std::basic_istream<char, std::char_tra
        + 1,73% std::basic_ostream<char, std::char_traits<char> >& std::operator<< <std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&, char const*)
       4,40% Patricia<unsigned long>::remove(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&)
     + 2,50% Patricia<unsigned long>::insert(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&, unsigned long const&)
       1,12% tolower
       0,89% Patricia<unsigned long>::cleanNode(Node<unsigned long>*) [clone .isra.0]
-   37,17%     9,15%  PATRICIA  libstdc++.so.6.0.33  [.] std::basic_istream<char, std::char_traits<char> >& std::operator>><char, std::char_traits<char>, std::allocator<char>
   + 28,02% std::basic_istream<char, std::char_traits<char> >& std::operator>><char, std::char_traits<char>, std::allocator<char> >(std::basic_istream<char, std::char_traits<ch
   + 4,02% 0x9800000000000011
+   35,94%     0,00%  PATRICIA  [unknown]            [.] 0x9800000000000011
+   24,18%     0,33%  PATRICIA  libstdc++.so.6.0.33  [.] std::istream& std::istream::_M_extract<unsigned long>(unsigned long&)
+   18,31%     7,24%  PATRICIA  libstdc++.so.6.0.33  [.] std::istreambuf_iterator<char, std::char_traits<char> > std::num_get<char, std::istreambuf_iterator<char, std::char_tr
+   16,93%     2,62%  PATRICIA  libstdc++.so.6.0.33  [.] std::istream::sentry::sentry(std::istream&, bool)
+   10,90%     1,87%  PATRICIA  libstdc++.so.6.0.33  [.] std::ostream::flush()
+   10,07%     5,50%  PATRICIA  libstdc++.so.6.0.33  [.] __gnu_cxx::stdio_sync_filebuf<char, std::char_traits<char> >::underflow()
+    8,74%     1,19%  PATRICIA  libc.so.6            [.] _IO_fflush
+    7,86%     7,86%  PATRICIA  libc.so.6            [.] ungetc
+    7,78%     0,56%  PATRICIA  libc.so.6            [.] _IO_file_sync@@GLIBC_2.2.5
+    7,15%     0,56%  PATRICIA  libc.so.6            [.] _IO_do_write@@GLIBC_2.2.5
+    6,45%     6,45%  PATRICIA  libc.so.6            [.] _IO_sputbackc
+    6,34%     6,34%  PATRICIA  libc.so.6            [.] _IO_getc
+    5,46%     5,46%  PATRICIA  libc.so.6            [.] __GI___libc_write
+    4,90%     3,56%  PATRICIA  PATRICIA             [.] Patricia<unsigned long>::insert(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&
+    4,78%     3,01%  PATRICIA  libstdc++.so.6.0.33  [.] __gnu_cxx::stdio_sync_filebuf<char, std::char_traits<char> >::uflow()
+    4,46%     4,29%  PATRICIA  PATRICIA             [.] Patricia<unsigned long>::remove(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&
+    4,02%     0,00%  PATRICIA  [unknown]            [.] 0000000000000000
+    3,43%     3,43%  PATRICIA  libstdc++.so.6.0.33  [.] getc@plt
+    3,34%     0,00%  PATRICIA  [unknown]            [.] 0x00007d40ad460036
+    3,34%     0,00%  PATRICIA  [unknown]            [.] 0x00007d40ad460037
+    3,31%     0,00%  PATRICIA  [unknown]            [.] 0x00007d40ad460034
+    3,28%     0,00%  PATRICIA  [unknown]            [.] 0x00007d40ad460061
+    3,21%     0,00%  PATRICIA  [unknown]            [.] 0x00007d40ad460063
+    3,21%     0,00%  PATRICIA  [unknown]            [.] 0x00007d40ad460064
+    3,19%     0,00%  PATRICIA  [unknown]            [.] 0x00007d40ad460038
```

Можем заметить, что большая часть времени (примерно 60%) выполнения тратится на библиотечный парсинг ввода. Это происходит,

потому, что потоки ввода-вывода («std::cin, std::cout») синхронизированы с потоками ввода-вывода языка «C» («scanf, printf»). Чтобы решить эту проблему, отключим синхронизацию с «C» потоками, а также отвяжем связь «cin» с «cout», чтобы избавиться от сбрасывания буфера «flush» перед каждым чтением.

Добавим изменения в функцию «main»:

```
int main() {

    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

...
```

Пересоберём проект и снова запустим утилиту «perf»

```
perf record -g -- ./PATRICIA < ../Discrete-analysis-
Labs/Laboratory_2/tests/test_mixed_1m.txt > /dev/null
```

```
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0,252 MB perf.data (3254 samples) ]
```

На данном этапе уже можем заметить, что количество сэмплов стало кратно ниже (примерно в 2.6 раза)

Откроем файл «perf.data»

```
perf report -g
```

```
Samples: 3K of event 'task-clock:upppH', Event count (approx.): 813500000
  Children      Self  Command   Shared Object         Symbol
-   64,35%    31,71%  PATRICIA  PATRICIA              [.] main
   - 32,64% main
      + 9,93% std::basic_istream<char, std::char_traits<char> >& std::operator>><char, std::char_traits<char>, std::allocator<char> >(std::basic_istream<char, std::char_traits<
      + 8,05% std::istream& std::istream::_M_extract<unsigned long>(unsigned long&)
      + 5,29% std::ostream& std::ostream::_M_insert<unsigned long>(unsigned long)
      + 4,64% std::basic_ostream<char, std::char_traits<char> >& std::operator<< <std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&, char const*)
        1,29% std::basic_ostream<char, std::char_traits<char> >& std::__ostream_insert<char, std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&, char c
        0,55% __strlen_avx2
   - 29,90% 0x9800000000000011
        main
-   44,13%     0,00%  PATRICIA  [unknown]             [.] 0x9800000000000011
   - 0x9800000000000011
      - 42,29% main
        + 5,29% std::ostream& std::ostream::_M_insert<unsigned long>(unsigned long)
        + 3,23% std::basic_istream<char, std::char_traits<char> >& std::operator>><char, std::char_traits<char>, std::allocator<char> >(std::basic_istream<char, std::char_trai
        + 1,26% std::basic_ostream<char, std::char_traits<char> >& std::operator<< <std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&, char const*)
          1,20% std::basic_ostream<char, std::char_traits<char> >& std::__ostream_insert<char, std::char_traits<char> >(std::basic_ostream<char, std::char_traits<char> >&, cha
        0,71% __memcmp_avx2_movbe
-   40,44%     0,00%  PATRICIA  [unknown]             [.] 0x0000000000000009
   - 0x9
      + 17,70% main
        10,36% Patricia<unsigned long>::remove(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&)
      + 5,84% Patricia<unsigned long>::insert(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&, unsigned long const&)
        2,64% tolower
        2,37% Patricia<unsigned long>::cleanNode(Node<unsigned long>*) [clone .isra.0]
-   13,09%     9,93%  PATRICIA  PATRICIA              [.] Patricia<unsigned long>::insert(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&
   - 10,42% Patricia<unsigned long>::insert(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&, unsigned long const&)
      - 2,92% operator new(unsigned long)
         - malloc
              2,15% _int_malloc
   - 1,48% 0x30
        0x9
        Patricia<unsigned long>::insert(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&, unsigned long const&)
+   10,45%    10,08%  PATRICIA  PATRICIA              [.] Patricia<unsigned long>::remove(std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> > const&
+    9,99%     4,73%  PATRICIA  libstdc++.so.6.0.33   [.] std::basic_istream<char, std::char_traits<char> >& std::operator>><char, std::char_traits<char>, std::allocator<char>
+    8,11%     0,37%  PATRICIA  libstdc++.so.6.0.33   [.] std::istream& std::istream::_M_extract<unsigned long>(unsigned long&)
+    6,85%     6,18%  PATRICIA  libstdc++.so.6.0.33   [.] std::istreambuf_iterator<char, std::char_traits<char> > std::num_get<char, std::istreambuf_iterator<char, std::char_tr
+    6,79%     0,00%  PATRICIA  [unknown]             [.] 0x0000000000000030
+    5,29%     0,80%  PATRICIA  libstdc++.so.6.0.33   [.] std::ostream& std::ostream::_M_insert<unsigned long>(unsigned long)
+    4,89%     4,89%  PATRICIA  libc.so.6             [.] tolower
+    4,70%     0,09%  PATRICIA  libstdc++.so.6.0.33   [.] std::basic_ostream<char, std::char_traits<char> >& std::operator<< <std::char_traits<char> >(std::basic_ostream<char,
+    4,64%     1,87%  PATRICIA  libstdc++.so.6.0.33   [.] std::basic_ostream<char, std::char_traits<char> >& std::__ostream_insert<char, std::char_traits<char> >(std::basic_ost
+    4,49%     4,43%  PATRICIA  libstdc++.so.6.0.33   [.] std::istream::sentry::sentry(std::istream&, bool)
+    3,47%     0,00%  PATRICIA  [unknown]             [.] 0000000000000000
+    2,98%     0,74%  PATRICIA  libc.so.6             [.] malloc
+    2,95%     0,12%  PATRICIA  libstdc++.so.6.0.33   [.] operator new(unsigned long)
```

Проанализировав результаты, можем заметить, что больше всего времени также тратится на ввод-вывод, а также функции удаления и вставки. Для более точного анализа проблемного места в коде, где наша функция тормозит. Для этого откроем функцию «remove» в режиме аннотации:

```
Samples: 3K of event 'task-clock:upppH', 4000 Hz, Event count (approx.): 813500000
Patricia<unsigned long>::remove(std::__cxx11::basic_string<char, std::char_traits<char>,
Percent
  1,22   60:   mov     %eax,%ecx
               mov     %esi,%r10d
  0,91         neg     %ecx
               and     $0x7,%ecx
               shl     %cl,%r10d
             Target = (bit == 0) ? Target->get_left() : Target->get_right();
  1,52         test    %r10b,(%rdi,%rdx,1)
             ↓ je      96
             Node<T>* get_right() {return right;}
  4,88         mov     0x38(%rbx),%rbp
             size_t get_index() {return index;}
  3,96         mov     0x28(%rbp),%rdx
             while (Target->get_index() > q->get_index()) {
 14,63         cmp     %rdx,%rax
             ↓ jae     a3
  2,44   80:   mov     %rbx,%r14
               mov     %rdx,%rax
               mov     %rbp,%rbx
             size_t byte_index = (index - 1) / 8;
         89:   lea     -0x1(%rax),%rdx
  0,30         shr     $0x3,%rdx
             if(byte_index >= key.length()) return 0;
  6,10         cmp     %r8,%rdx
             ↑ jb      60
             Node<T>* get_left() {return left;}
  5,49   96:   mov     0x30(%rbx),%rbp
             size_t get_index() {return index;}
  3,96         mov     0x28(%rbp),%rdx
             while (Target->get_index() > q->get_index()) {
 21,95         cmp     %rdx,%rax
             ↑ jb      80
             if (root == nullptr) return false;
         a3:   xor     %ecx,%ecx
  0,30         cmp     0x8(%rbp),%r8
             ↓ je      140
             Target->set_value(q->get_value());
             Target->set_key(q->get_key());
             delete q;
             return true;

             }
         af:   add     $0x38,%rsp
               mov     %ecx,%eax
               pop     %rbx
               pop     %rbp
```

Можно заметить, что основные операции, вызывающие торможение, являются «cmp (*compare – сравнение*) %rax,%rdx» эти команды отвечают за

сравнение двух элементов, которые процессору нужно достать из оперативной памяти («mov»). Так как кеш процессора гораздо быстрее оперативной памяти, процессору приходится ждать до 100-200 тактов, каждый раз, когда происходит спуск по дереву.

Выводы о найденных недочётах

В ходе выполнения лабораторной работы, была найдена проблема производительности, которая заключалась в долгой работе операций ввода-вывода. Проблема была устранена отключением синхронизации с выводом из языка «C» и отключению связи «cin» и «cout», чтобы убрать частые сбросы буфера.

Профилирование через «perf report» и «perf annotate» показало, что основным узким местом является спуск по дереву (строки с «while(current->get_index() > ...)»). Ассемблерные инструкции сравнения занимают больше всего времени не из-за математической сложности, а из-за задержек обращения к памяти. Узлы дерева неравномерно распределены на куче, поэтому процессору необходимо ожидать данные из оперативной памяти.

Для решения проблемы частого обращения к оперативной памяти, можно использовать один большой кусок памяти, где каждому узлу будет выдаваться адрес его места на этом куске. К примеру, размещать узлы на «std::vector». Это позволит также проще отлаживать программу и избежать ошибок ручной работы с памятью.

Сравнение работы исправленной программы с предыдущей версией.

После исправления проблемы долгого ввода-вывода, программа стала работать примерно в 2.5 раза быстрее, что можно наблюдать из результатов тестирующей платформы, а также из отчёта профилирования программы.

Время посылки	ID	Задача	Компилятор	Вердикт	Тип посылки	Время	Память	Тест	Баллы	
12 апр 2026, 13:12:34	160272224	3	C++20 (GCC 14.1)	OK	—	2.972s	223.01Mb	—	—	отчёт
12 апр 2026, 00:08:31	160258139	3	C++20 (GCC 14.1)	OK	—	7.574s	223.01Mb	—	—	отчёт

Выводы

В результате выполнения данной лабораторной работы, была исследована структура PATRICIA, реализованная в лабораторной работе №2. В ходе выполнения были изучены инструменты тестирования на скорость и стабильность. Для исследования ошибок памяти, использовались санитайзеры и «valgrind». Для оценки скорости выполнения программы использовалась утилита «perf». В результате исследования, были найдены и исправлены недочёты, которые позволили ускорить программу в 2.5 раза. Также была разобрана и обоснована проблема скорости выполнения, связанная с особенностью размещения узлов на куче.

Из проблем, которые возникли в момент выполнения лабораторной работы, можно отметить, сложность установки утилиты «perf» под WSL (Windows Subsystem for Linux), так как в ходе исследования, команды «perf stat», было необходимо включить обработку «PMU» (Performance Monitoring Unit) – блок мониторинга производительности, которые позволяют использовать аппаратные счётчики производительности ЦП. Эти счётчики оказались недоступными в данный момент времени для процессоров Intel 12-го поколения и выше. Разработчики WSL уже работают над решением этой проблемы, но несмотря на отсутствие доступа к аппаратным счетчикам, тесты были успешно проведены с использованием программных событий. В качестве замены использовались программные события.

Данная лабораторная работа помогла разобраться в большом количестве новых инструментов тестирования и исследования ошибок программы, что позволило приобрести полезный опыт.